

qt_auto_translation

This tool provides an automated pipeline to translate Qt `.ts` files using an AI-based translation API. It supports batch translation, CSV export/import, and dynamic creation of `.ts` files for new languages.

Features:

- AI-based translation of Qt `.ts` files
- CSV export/import of translation data for manual editing or localization teams
- Clear translations option to reset all entries
- Dynamic creation of `.ts` files from a template with custom language/postfix
- Context-aware translations using `<context><name>` metadata
- Configurable through a single JSON file
- Simple GUI

Structure:

The entire project is inside the `qt_auto_translation` folder. One below inside that folder, the following are placed:

- `auto_translator_cfg.json` (a config file where all the decisions are made)
- `main.cpp` (where all the functions are called and used)
- `language_en_US.ts` the initial `.ts` file created by Qt linguistic
- `CMakeLists.txt`
- `translation` (folder)
 - `CMakeLists.txt`
 - `translation.cpp` (where all the translation functions are defined)
 - `translation.h` (where all the translation functions are declared)
- `translatorGUI`
 - `CMakeLists.txt`
 - `mainwindow.cpp` (where the GUI logics are created and linked to the GUI components)
 - `mainwindow.h` (where the GUI logics are declared)
 - `mainwindow.ui` (where the GUI is designed)
 - `main.cpp` (where the GUI is launched)

Config File:

The `auto_translator_cfg.json` file is where all the options are controlled. Here is an example:

```
{
  "api_call_size": 100,
  "api_key_path": "qt_auto_translation/secrets.txt",
  "clear_translation": false,
  "csv_to_export": "test.csv",
  "csv_to_import": "test.csv",
  "export_to_csv": true,
  "import_from_csv": false,
  "lang": "French",
  "lang_postfix": "fr_CA",
  "template_ts_file": "qt_auto_translation/language_en_US.ts",
  "ts_file_path": "",
  "write_back_to_ts": true
}
```

api_call_size	Number of strings to send per batch when calling the translation API.
api_key_path	Path to the text file containing the API key.
clear_translation	If true, clear all existing translations in the .ts file before proceeding.
csv_to_export	Path where the tool writes translations to a CSV file (if export_to_csv is true).
csv_to_import	Path from which translations will be imported (if import_from_csv is true).
export_to_csv	If true, save the current translation data to a CSV file after processing.
import_from_csv	If true, skips API translation and instead populates translations from the provided CSV.
lang	The name of the target language (e.g., "French", "Turkish").
lang_postfix	The Qt-compatible locale code used in filenames (e.g., "fr_CA", "tr_TR").
template_ts_file	Path to the source .ts file containing the original English (or base language) strings.

ts_file_path	Path to the output .ts file where translations will be written. If empty, it will be auto-generated using the postfix.
write_back_to_ts	If true, writes translated content into the .ts file. If false, only CSV export happens.
Figure 1: Config Fields	

GUI:

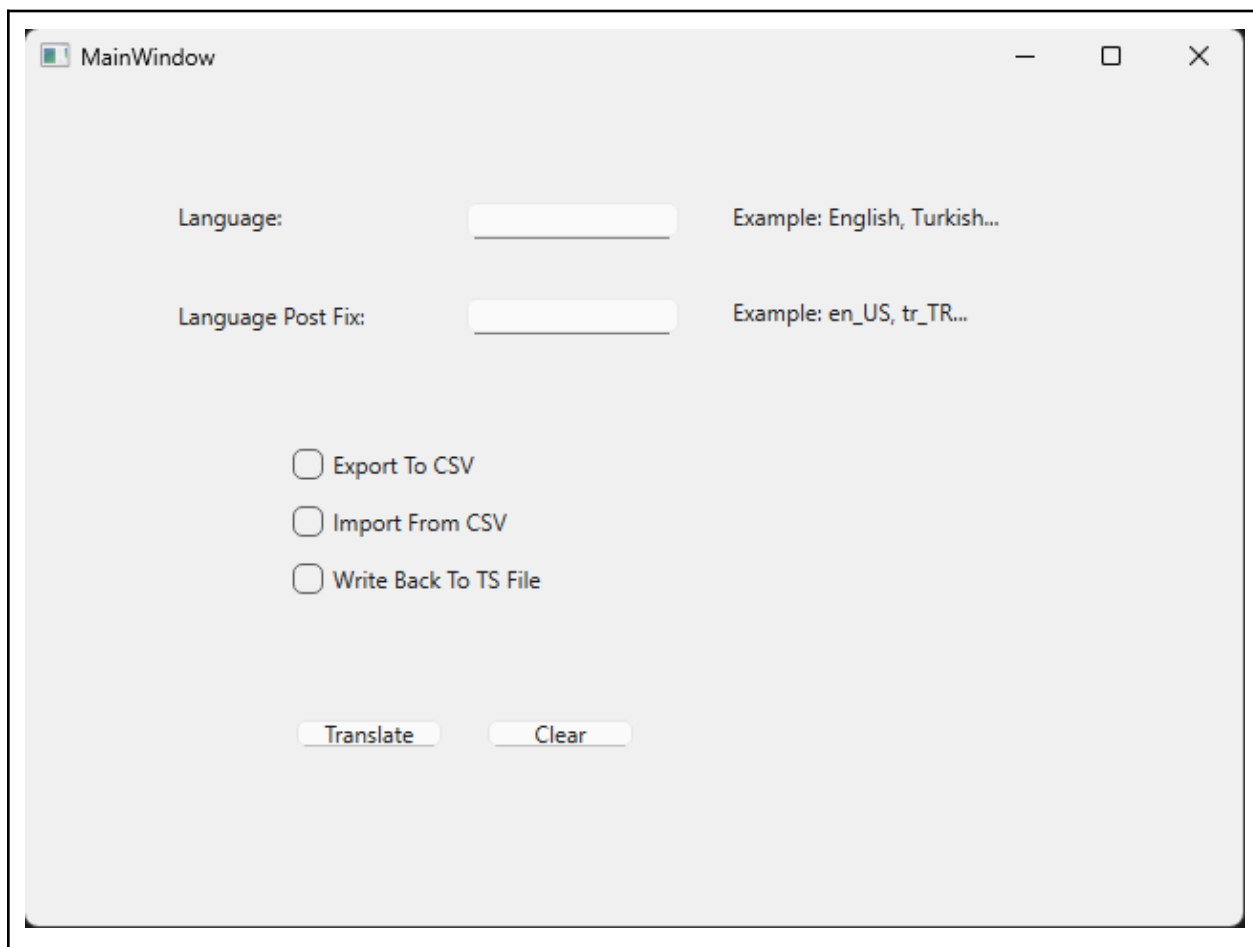


Figure 2: GUI

How To Use:

1. Fill out the config file with the correct paths
2. Run translatorGUI.exe
3. Enter the language that you wish to translate to in the language field
4. Enter the language postfix that will be used to create the .ts file
5. Set the checkboxes according to your preferences
6. Press "Translate"

If you check "Export To CSV", the data will be exported to a CSV file. You can later change the translation according to your preferences in the CSV file and run the program once again with "Import From CSV" and "Write Back To TS File" checked to push your own translations to the .ts file.

You can clear the existing translations in the .ts file by simply entering the correct language and postfix, checking the "Write Back To TS File" box and pressing "Clear".

Errors:

Error Message	Cause	Fix
<i>Unable to open file</i>	.ts or .csv file path is invalid or file doesn't exist.	Check the <code>ts_file_path</code> , <code>csv_to_export</code> , or <code>csv_to_import</code> in your config. Ensure the file exists or is created properly.
<i>API key is empty or could not be read</i>	The <code>secrets.txt</code> file path is wrong or file is empty.	Verify the <code>api_key_path</code> in your config and ensure the file contains your OpenAI API key.
<i>Template TS file does not exist</i>	The .ts file doesn't exist yet and the <code>template_ts_file</code> path is invalid.	Ensure that <code>template_ts_file</code> points to a valid .ts template file.
<i>Context not found</i>	The CSV file includes a context that doesn't exist in the .ts file.	Ensure the context names in the CSV match the source .ts file exactly.
<i>Message with source not found in context</i>	Source string in CSV not found in the specified context.	CSV may be outdated or mismatched. Re-export with updated .ts data.

<i>Failed to parse API response as JSON. or Failed to parse the returned translation JSON.</i>	GPT API didn't return valid JSON (e.g., formatting issues).	Check API key validity, internet connection, and ensure GPT model supports your usage. You may want to log the raw API response.
<i>Failed to start translation executable</i>	Path to <code>qt_auto_translation.exe</code> is incorrect or binary missing.	Check the build path used in GUI and ensure the CLI binary is compiled and in place.
Figure 3: Error List		